`phylo`: Technical Document

M. J. Wilson

September 2, 2026

Abstract

This document states the science behind `phylo`: the substitutionmodel, the likelihood and the algorithm that evaluates it, the tree-searchproblem and its move sets, the reinforcement-learning formulation layered ontop, and the computational structure that makes any of it tractable. It alsoholds the captions for the figures and tables the quality-assurance suiteemits, so that a plot and its description cannot drift apart. Equations andalgorithms follow the formulations in the sources cited throughout;deviations are stated where they occur.

Contents

1 Scope and status **2**

2 Notation **2**

3 The substitution model **2**
3.1 Reversibility and the root distribution 3
3.2 The k-state Jukes–Cantor model 3

4 The likelihood **3**
4.1 Pruning 3
4.2 Differentiable backends 4
4.3 Rate variation across sites 4

5 Simulation **4**

6 Tree search **5**
6.1 The size of the problem 5
6.2 Move sets 5
6.3 Canonical serialization 6
6.4 Extensions to the move set 6
6.5 Parsimony and likelihood 7

7 Gradients **7**

8 Reinforcement-learning formulation **8**

9 Computational structure **8**

10 Quality assurance **8**

11 Sources **10**
11.1 Infrastructure: build, structure, and speed 10
11.2 Optimization: discrete and continuous 10
11.3 Application: the science 11

1 Scope and status

The project's goal is to search the space of phylogenetic tree topologieswith reinforcement learning, scoring candidates by their likelihood under amodel of character substitution. This document develops the theory thatsearch requires.Most of it is not implemented yet. The k-state Jukes–Cantor substitutionmodel, its simulator, and vectorized NumPy Felsenstein pruning (the Pruningsubsection, `phylo.likelihood.pruning`) are; the rest of therepository is scaffolding — build, test, benchmark, lint, documentation,and audit pipelines around placeholder functions — and this document iswritten ahead of that implementation so it has a specification to match.Sections marked *not yet implemented* describe intent; everything elsedescribes mathematics that does not depend on our code.

2 Notation

Let $\mathcal{A}$ be the finite state space of a single character, with$k = |\mathcal{A}|$; for nucleotide data $\mathcal{A} = \{A, C, G, T\}$ and $k = 4$.The data $\mathbf{D}$ are an alignment of n taxa over m sites, so$\mathbf{D} \in \mathcal{A}^{n \times m}$, with $\mathbf{D}_{\cdot s}$ the observed columnat site s.A phylogeny is a pair $(\tau, \mathbf{t})$: a tree topology τ whose$n$ leaves carry the taxa, and a vector $\mathbf{t}$ of non-negative branchlengths, measured in expected substitutions per site. Internal nodesrepresent unobserved ancestors. Table 1 collects the symbolsused throughout.

Symbol	Meaning
$\mathcal{A}$, k	character state space and its size
n, m	number of taxa, number of sites
τ	tree topology
$\mathbf{t}$	vector of branch lengths
$\mathbf{Q}$	instantaneous rate matrix, $k \times k$
$\mathbf{P}(t)$	transition-probability matrix over a branch of length t
$\boldsymbol{\pi}$	root (and, under reversibility, stationary) distribution
$L_u(i)$	partial likelihood at node u given state i

Table 1: Symbols used in this document.

3 The substitution model

Character evolution along a branch is a continuous-time Markov chain on$\mathcal{A}$ with rate matrix $\mathbf{Q}$, whose off-diagonal entries $q_{ij} \geq 0$give the instantaneous rate of substitution from state i to state j andwhose rows sum to zero,

$$q_{ii} = -\sum_{j \neq i} q_{ij}. \tag{1}$$

Transition probabilities over a branch of length t follow from the matrixexponential,

$$\mathbf{P}(t) = \exp(\mathbf{Q}t), \qquad P_{ij}(t) = \Pr(X_t = j \mid X_0 = i). \tag{2}$$

$\mathbf{P}(t)$ is a stochastic matrix: $P_{ij}(t) \geq 0$ and each row sums toone. Both properties are invariants the implementation is tested against.

3.1 Reversibility and the root distribution

The models used here are time-reversible with respect to a distribution$\boldsymbol{\pi}$, meaning they satisfy detailed balance,

$$\pi_i q_{ij} = \pi_j q_{ji} \quad \text{for all } i, j. \tag{3}$$

Reversibility has two consequences we rely on. First, $\boldsymbol{\pi}$ is thestationary distribution of the chain, which makes it the natural choice forthe distribution of states at the root. Second, the likelihood is invariantto where the root is placed on the tree — Felsenstein's *pulleyprinciple* [8] — so an unrooted topology suffices forinference, and re-rooting is a test the implementation must pass rather thana modelling decision.A general reversible rate matrix factorizes as

$$q_{ij} = s_{ij}\pi_j \quad (i \neq j), \qquad s_{ij} = s_{ji}, \tag{4}$$

which is the general time-reversible (GTR) model; the familiar JC69, K80, andHKY85 models are constrained special cases [8]. Branchlengths measure expected substitutions per site, which fixes the scale of$\mathbf{Q}$ through the normalization

$$-\sum_i \pi_i q_{ii} = 1. \tag{5}$$

Without (5), $\mathbf{Q}$ and $\mathbf{t}$ trade off against eachother and neither is identifiable.

3.2 The k-state Jukes–Cantor model

The most constrained case is worth stating in closed form, because it givesthe simulator an oracle that owes nothing to our likelihood code. Take allexchange rates equal and $\boldsymbol{\pi}$ uniform on k states. Under thenormalization (5), the transition probabilities over abranch of length t are

$$P_{ii}(t) = \frac{1}{k} + \frac{k-1}{k}\, e^{-kt/(k-1)}, \qquad P_{ij}(t) = \frac{1}{k} - \frac{1}{k}\, e^{-kt/(k-1)} \quad (i \neq j). \tag{6}$$

At $t = 0$ this gives the identity, and as $t \to \infty$ every entry tendsto $1/k$, the stationary distribution. For $k = 4$ it is JC69[8]. Simulated substitution frequencies are checkedagainst (6) within Monte Carlo error, which tests the simulatorindependently of the pruning implementation — if both were validatedagainst each other, a shared error would pass unnoticed.

4 The likelihood

Sites are modelled as independent given $(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$, so thelog-likelihood is a sum over columns,

$$\log \mathcal{L}(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi} \mid \mathbf{D}) = \sum_{s=1}^{m} \log \Pr(\mathbf{D}_{\cdot s} \mid \tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi}). \tag{7}$$

The site term marginalizes over the unobserved states of every internal node.Written directly, that is a sum over k^{n-1} assignments — intractablebeyond a handful of taxa. Felsenstein's pruning algorithm computes it in timelinear in n by exploiting the tree's conditional independence structure[8]; it is the sum-product algorithm on a tree[14, 12].

4.1 Pruning

Define the partial likelihood $L_u(i)$ as the probability of the data at theleaves below node u, given that u is in state i. At a leaf withobserved state a, $L_u(i) = \mathbb{1}[i = a]$. At an internal node $u$with children v joined by branches of length t_v,

$$L_u(i) = \prod_{v \in \text{children}(u)} \left(\sum_{j \in \mathcal{A}} P_{ij}(t_v)\, L_v(j) \right), \tag{8}$$

and at the root r the site likelihood is

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \sum_{i \in \mathcal{A}} \pi_i \, L_r(i). \tag{9}$$

Algorithm 1 costs $O(nk^2)$ per site against $O(k^{n-1})$for direct marginalization. The two agree

Algorithm 1 Felsenstein pruning for one site

Require: topology τ, branch lengths $\mathbf{t}$, rate matrix $\mathbf{Q}$,root distribution $\boldsymbol{\pi}$, observed column $\mathbf{D}_{\cdot s}$
Ensure: site likelihood $\Pr(\mathbf{D}_{\cdot s} \mid \tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$

1: **for all** nodes u in post-order **do**
2: **if** u is a leaf with observed state a **then**
3: $L_u(i) \leftarrow \mathbb{1}[i = a]$ for all $i \in \mathcal{A}$
4: **else**
5: $L_u(i) \leftarrow \prod_{v \in \text{children}(u)} \sum_j P_{ij}(t_v) L_v(j)$ for all $i \in \mathcal{A}$
6: **end if**
7: **end for**
8: **return** $\sum_i \pi_i L_r(i)$

exactly, which makes brute-forcemarginalization on small trees the reference implementation the pruning codeis tested against.Partial likelihoods underflow for realistic m and n, so theimplementation will rescale them per node and accumulate the log of thescaling factors. Rescaling changes the arithmetic but not the result: therescaled and direct computations must agree to within floating-pointtolerance on small problems where both run.

4.2 Differentiable backends

Branch lengths $\mathbf{t}$ are what gradient-based fitting and tree searchdifferentiate through, so an accelerated backend must keep them in theautodiff graph rather than baking them into topology state. The PyTorchbackend (`pruning_torch.py`, float64, CPU) takes $\mathbf{t}$ as atensor separate from the topology and is accepted when its log-likelihoodagrees with the NumPy reference to atol $= 10^{-9}$ – the samebound as the brute-force check above – never relaxed to accommodate adiscrepancy. Gradients obtained from `torch.autograd` are checkedagainst central finite differences of the NumPy oracle (step$\epsilon = 10^{-6}$) to atol $= 10^{-6}$, the coarser bound set bythe finite-difference truncation and rounding error at that step size, notby the autodiff computation itself.

4.3 Rate variation across sites

Sites evolve at different rates. The standard treatment draws asite-specific rate multiplier r from a discretized gamma distribution withmean one, and averages the site likelihood over the c rate categories,

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \frac{1}{c} \sum_{g=1}^{c} \Pr(\mathbf{D}_{\cdot s} \mid \tau, r_g \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi}), \tag{10}$$

multiplying the cost of a likelihood evaluation by c[8]. *Not yet implemented.*

5 Simulation

Simulation is how the implementation is tested: draw data from knownparameters, then check that each component recovers what the generating modelimplies. Generation runs in pre-order, the reverse of pruning.Every draw uses an explicitly seeded generator, so a simu-

Algorithm 2 Simulating an alignment under $(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$

Require: topology τ, branch lengths $\mathbf{t}$, rate matrix $\mathbf{Q}$,root distribution $\boldsymbol{\pi}$, sites m, seeded generator
Ensure: alignment $\mathbf{D} \in \mathcal{A}^{n \times m}$
1: **for** $s \leftarrow 1$ to m **do**
2: draw the root state $x_r \sim \boldsymbol{\pi}$
3: **for all** nodes u in pre-order, $u \neq r$ with parent p and branch t_u **do**
4: draw $x_u \sim \mathbf{P}(t_u)_{x_p,\cdot}$
5: **end for**
6: record x_u at every leaf u as column s
7: **end for**
8: **return D**

lated dataset isreproducible from its seed. Implemented for the k-state Jukes–Cantor casein `phylo.sim` (`simulate.py`), with topology, branch lengths,k, $\boldsymbol{\pi}$, seed, and site count read from a `simulation_params.yaml`and the generating parameters retained alongside the alignment. Simulatedsubstitution frequencies are checked against (6) within ayaml-declared Monte Carlo tolerance in`tests/regression/test_jc_simulate.py`.Table 3 in Section 10 lists everysimulation regression fixture at the problem sizes actually run, readdirectly from each fixture's yaml; Figure 2 there showsa complete generated instance at the smallest of them — the Newick stringand the leading simulated sites of the alignment.

6 Tree search

6.1 The size of the problem

The number of distinct unrooted binary topologies on n taxa is the doublefactorial $(2n-5)!!$ [24], which grows faster than exponentially(Table 2). Exhaustive search is impossible past a handfulof taxa, and finding the optimal tree under either the parsimony or thelikelihood criterion is NP-hard [8]. Practical inferenceis therefore heuristic: start somewhere, propose rearrangements, keep whatscores better.

Taxa n	Unrooted binary topologies $(2n-5)!!$
10	2.0×10^{6}
20	2.2×10^{20}
50	2.8×10^{74}

Table 2: Growth of tree space with the number of taxa.

6.2 Move sets

Two rearrangement neighbourhoods define the search graph[8]:

NNI (nearest-neighbour interchange). Each internal edge of anunrooted binary tree separates four subtrees, which can be rejoined inthree ways; two are new topologies. With $n-3$ internal edges, the NNIneighbourhood contains $2(n-3)$ topologies — small, cheap, and local.

SPR (subtree prune and regraft). Detach a subtree and reattach it atanother edge. The neighbourhood is quadratic in n, so each step costsmore, but the larger reach escapes local optima that trap NNI.

Since neighbouring topologies differ only near the affected edges, mostpartial likelihoods are unchanged between a tree and its neighbour. Cachingand recomputing only the invalidated path is what makes tree searchaffordable, and is a design requirement on the likelihood engine rather thanan optimization to add later.

6.3 Canonical serialization

Newick notation is not a unique encoding: one topology admits many strings,differing by the order in which children are written and by where thestring is rooted. A search that keys on the raw string therefore fails torecognize a tree it has already scored, and a model trained on raw stringslearns to distinguish spellings that carry no phylogenetic meaning.Algorithm 3 removes both problems by fixing arepresentative.Rooting at the smallest label is well defined

Algorithm 3 Canonical form of an unrooted binary tree with labelled leaves

Require: tree τ with leaves labelled from a totally ordered set
Ensure: a string identifying τ's topology uniquely
root τ at the leaf whose label is smallest in that order
for all nodes u in post-order **do**
    if u is a leaf **then**
        $\kappa(u) \leftarrow \text{label}(u)$
    else
        sort the children v of u by $\kappa(v)$
        $\kappa(u) \leftarrow$ the concatenation of $\kappa(v)$ in that order
    end if
end for
return κ(root), emitting children in sorted order

for any labelled tree, and thechild order is fixed bottom-up by keys that depend only on the subtreebelow, so the output depends on the topology alone. Hashing the keys keepsthe cost at $O(n \log n)$ [6]. Branch lengths are carriedseparately, as a vector in canonical node order, so one topology under twolength assignments shares a topology key — which is what makes the keyusable for memoizing likelihood evaluations across a search.Two properties are required of any implementation, and both are testedrather than assumed: two trees produce the same string exactly when they arethe same topology, checked by randomly re-rooting and permuting the childrenof one tree and by comparing against distinct trees; and parsing a canonicalstring reproduces the tree that produced it.

6.4 Extensions to the move set

Three extensions target the same bottleneck: the number of likelihoodevaluations a search spends. All three are proposals. *Not yetimplemented.*

Bounding whole sets of trees. Scoring every neighbour exactlywastes work when most are poor. A cheap *upper* bound on the likelihoodattainable anywhere within a set of topologies lets the search discard thewhole set, in the manner of branch and bound [6]: when thebound falls below the best log-likelihood already achieved, no member of theset can win. Two sources of bounds are worth trying: relaxations of thepruning recursion (8), which replace the product overchildren by a quantity that is cheaper and provably no smaller; andcompressed summaries of the alignment, where identical site patterns alreadycollapse and coarser encodings trade tightness for speed[2]. Soundness is the whole game — a bound that can fallbelow an attainable likelihood silently discards the optimum — so eachcandidate bound needs a proof, and a test that searches small treesexhaustively for a counterexample.

Multi-moves. Composing several NNI or SPR rearrangements intoone action reaches topologies no single move reaches, at the cost of a muchlarger action space; in reinforcement-learning terms these are temporallyextended actions [25]. Rather than fixing how manyrearrangements a compound move contains, draw the composition duringtraining from a Dirichlet process, so the number of distinct compound movesthe agent maintains grows with the evidence for them instead of being set inadvance [14]. The measure of success is unchanged: fewerlikelihood evaluations for the same likelihood reached.

A transformer policy over Newick strings. A topology serializesto a Newick string, which a transformer can consume directly — tokenizethe string, pretrain on simulated trees, then fine-tune the policy with theusual policy-gradient machinery[23, 13, 25]. The input is the canonicalform of Section 6.3, which settles the encoding question:one topology, one string. The alternative, an architecture invariant tochild order and root placement, buys the same property at greater cost andwithout supplying the memoization key the search needs anyway. Compressionthen applies on top, replacing recurring subtrees by dictionary symbols —an extended alphabet over the canonical string [2] — whichshortens the token sequence a transformer must attend over.

6.5 Parsimony and likelihood

The *large parsimony* problem asks for the topology minimizing thenumber of state changes needed to explain the data. We keep the searchproblem and replace the criterion: candidates are scored by the maximizedlog-likelihood of (7), with the continuousparameters $(\mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$ fitted per topology. Likelihood is the moreprincipled criterion — it models the substitution process rather thancounting changes, and it is statistically consistent where parsimony can bepositively misleading [8] — at the cost of a far moreexpensive score.

7 Gradients

Fitting $(\mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$ for a candidate topology is a continuousoptimization, and gradients make it tractable. Differentiating(2) with respect to a branch length gives

$$\frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{Q}\exp(\mathbf{Q}t) = \mathbf{Q}\,\mathbf{P}(t). \tag{11}$$

For a reversible $\mathbf{Q}$ with eigendecomposition $\mathbf{Q} = \mathbf{U}\Lambda\mathbf{U}^{-1}$, both the exponential and its derivative are diagonal in theeigenbasis,

$$\mathbf{P}(t) = \mathbf{U}e^{\Lambda t}\mathbf{U}^{-1}, \qquad \frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{U}\Lambda e^{\Lambda t}\mathbf{U}^{-1}, \tag{12}$$

so a single decomposition serves every branch. Gradients of thelog-likelihood with respect to all branch lengths follow from reverse-modeautomatic differentiation through the pruning recursion(8), at a cost proportional to one forward evaluation[10, 22]. The optimizer consuming them is astandard constrained quasi-Newton or trust-region method[19]; the parameters carry constraints — non-negativebranch lengths, a normalized $\boldsymbol{\pi}$ — so the geometry of the statistical-manifold, and the natural gradient it induces, is the principled setting[1].Analytic and automatic gradients are checked against central finitedifferences,

$$\frac{\partial f}{\partial \theta} \approx \frac{f(\theta + h) - f(\theta - h)}{2h}, \tag{13}$$

with h chosen to balance truncation against round-off. This check is arequired test for any differentiable component. *Not yet implemented.*

8 Reinforcement-learning formulation

Classical heuristics propose moves by a fixed, hand-designed rule. We askwhether a learned policy proposes better ones, and cast the search as aMarkov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$[25]:

State. The current topology with its fitted continuous parameters,together with a summary of the alignment.

Action. A move drawn from the NNI or SPR neighbourhood of thecurrent topology.

Transition. Applying the move and refitting the continuousparameters — deterministic given the action, up to the optimizer.

Reward. The improvement in maximized log-likelihood,$R = \log \mathcal{L}(\tau') - \log \mathcal{L}(\tau)$.

Episode. One search trajectory from a starting tree, terminating ona step budget or when no move improves the score.

The natural performance measure is not the likelihood reached but thelikelihood reached *per likelihood evaluation*: every method convergesgiven unbounded evaluations, so the comparison against a hill-climbingbaseline has to be made at equal budget. *Not yet implemented.*

9 Computational structure

Likelihood evaluation dominates the run time: every proposed move costs atleast one evaluation, and search proposes many. Its structure is favourable— sites are independent by (7), and the per-nodework in (8) is a small dense matrix–vector product — so thecomputation is a batched, data-parallel reduction over a tree.The project's performance rule follows from that. Where a kernel plausiblyachieves an order of magnitude over the vectorized NumPy reference at thesizes actually run, it targets the GPU through PyTorch, Triton, or JAX, whichsupply autodiff and batching alongside the parallelism; otherwise the workgoes to the Rust backend. The threshold is a measured benchmark, not anexpectation, and every accelerated kernel keeps its reference implementationas a pinned oracle. Kernel-level considerations — memory hierarchy,occupancy, parallel reduction — follow Hwu et al. [11]; the CPU-sidecounterparts, caching and where time actually goes, followBryant and O'Hallaron [4], and the backend itself is Rust [3].Memory is the constraint to watch: partial likelihoods occupy$O(n\,m\,k\,c)$ floating-point values for n taxa, m sites, $k$states, and c rate categories, before any caching of unchanged subtreesacross proposals.

10 Quality assurance

The test suite is expected to establish scientific validity, not to exercisesyntax: fixtures come from component-wise simulation under known parameters,expected values come from analytic or brute-force sources, and the invariantsstated above — stochasticity of $\mathbf{P}(t)$, detailed balance(3), agreement of pruning with directmarginalization, gradients against finite differences (13),re-rooting invariance — are checked directly.Part of that suite emits scientific-style figures and tables: parameterrecovery against simulated truth, likelihood surfaces, convergence traces,and timing comparisons. **Their captions belong in this section**, addedin the same change that adds the figure, so that a plot and its descriptioncannot drift apart.Figure 1 is the first: the topology and branch lengths`phylo.sim.simulate` draws an alignment over, rendered by`phylo.qa.sim_tree` from the same 8-taxon regression fixture as`tests/regression/test_jc_simulate.py`.`infra/build_technical_doc.sh` regenerates the figure and thecaption above — read verbatim from the file the script writes alongsideit — on every build, so the two cannot drift apart.Table 3 tabulates the taxa count, site count,seed, and Monte Carlo tolerance of every simulation regression fixture,rendered

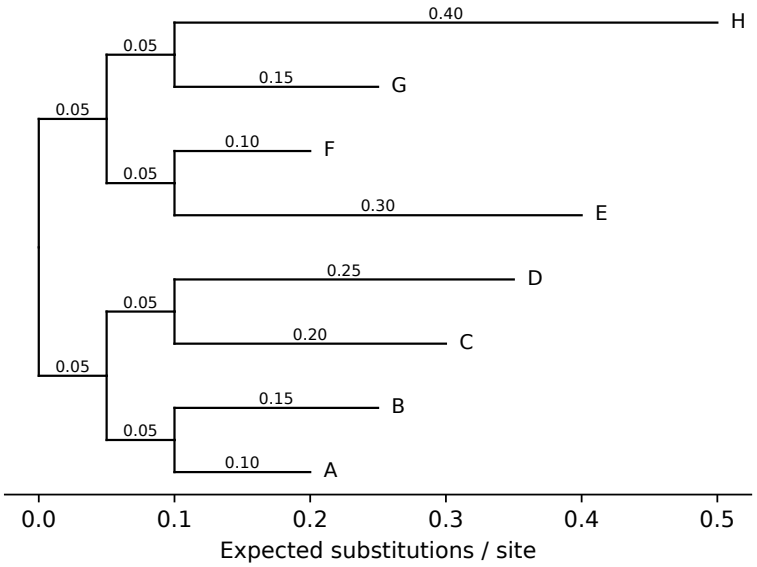

Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site.

by `phylo.qa.sim_problem_sizes` directly from the three`simulation_params.yaml` files rather than restated by hand, so thenumbers in this document cannot drift from what the regression suiteactually runs.Figure 2 shows a complete generated instance at thesmallest of those

Fixture	Taxa	Sites	Seed	Tolerance
simulation_params.yaml	4	200000	20260902	0.01
simulation_params_small_sites.yaml	4	20000	20260903	0.03
simulation_params_8taxa.yaml	8	200000	20260904	0.01

Table 3: Problem-size parameters across the 3 simulation regression fixtures backing the Jukes-Cantor simulation test: taxa count, site count, seed, and the Monte Carlo tolerance each validates simulated substitution frequencies within.

sizes: the Newick string `phylo.sim.newick.to_newick`produces for the 4-taxon fixture, and the leading simulated sites of thealignment, nucleotide-coded for this $k = 4$ model, rendered by`phylo.qa.sim_example`.Likelihood, optimization, and search QA figures are not built yet; thissection is where their captions go once they are.

```
(A:0.1,B:0.25,(C:0.15,D:0.4)ancestor_CD:0.05)root;
```

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 2: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200000 sites total), showing the generated Newick topology and the first 10 of 200000 simulated sites.

11 Sources

The texts this project works from, grouped as `CLAUDE.md` groups them— a routing table keyed on what each informs, rather than a reading list.Where an algorithm we implement appears in one of them, this documentfollows its formulation and notation and says where it deviates.

11.1 Infrastructure: build, structure, and speed

Software craft. Martin [15] on the structure that keeps achange reviewable; Blandy et al. [3] on ownership, lifetimes, and theFFI boundary the Rust backend crosses.

Systems and hardware. Bryant and O'Hallaron [4] on the memory hierarchyand where CPU time goes; Hwu et al. [11] on GPU kernels, occupancy,and parallel reduction and scan.

11.2 Optimization: discrete and continuous

Algorithms and discrete mathematics. Cormen et al. [6] on datastructures, traversal, complexity, and branch and bound;Rosen [24] on the counting and graph theory behind tree space.

Numerical optimization. Nocedal and Wright [19] on line search, trustregion, and quasi-Newton methods for the continuous parameters.

Probabilistic inference. MacKay [14] on inference, messagepassing, and Monte Carlo; Koller and Friedman [12] on factor graphs and theexact-inference machinery that (8) instantiates;Frey [9] on belief propagation read across inference andcoding; Ortega [20] on spectral methods for signals indexed bygraph vertices.

Statistical physics. Mézard and Montanari [16] on inference over sparsegraphs and phase transitions in hard combinatorial problems — treesearch being one; Newman and Barkema

[17] on sampling, equilibration, anderror estimates from correlated samples, which the benchmark harnessneeds to rank variants rather than noise.

Information geometry. Amari [1] on the statisticalmanifold, the Fisher metric, and natural gradient.

Learning and reinforcement learning. Goodfellow et al. [10] andPrince [22] on architectures, optimization, and automaticdifferentiation; Sutton and Barto [25] on MDPs, value and policymethods, and temporally extended actions; Lapan [13] onimplementation practice; Raschka [23] on transformer internalsand tokenization, for the policy over canonical Newick strings proposedin Section 6.4.

11.3 Application: the science

Phylogenetics. Felsenstein [8] on substitution models,pruning, tree search, and the parsimony and likelihood criteria;Durbin et al. [7] on probabilistic models of sequences;Compeau and Pevzner [5] on the algorithmic view of alignment and phylogenyconstruction; Pachter and Sturmfels [21] on the algebraic structure ofphylogenetic models and their invariants, which offer topology teststhat avoid likelihood optimization altogether.

Information and quantum. Blahut [2] on algebraic coding,which is the background for the compressed tree encodings inSection 6.4; Nielsen and Chuang [18] on quantuminformation and algorithms, background only — nothing here targetsquantum hardware.

References

[1] Shun-ichi Amari. *Information Geometry and Its Applications.* Springer, Tokyo, 2016.

[2] Richard E. Blahut. *Algebraic Codes for Data Transmission.* Cambridge University Press, Cambridge, 2003.

[3] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development.* O'Reilly Media, 2nd edition, 2021.

[4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective.* Pearson, 3rd edition, 2015.

[5] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach.* Active Learning Publishers, 2015.

[6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms.* MIT Press, Cambridge, Massachusetts, 4th edition, 2022.

[7] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteinsand Nucleic Acids.* Cambridge University Press, Cambridge, 1998.

[8] Joseph Felsenstein. *Inferring Phylogenies.* Sinauer Associates, Sunderland, Massachusetts, 2004.

[9] Brendan J. Frey. *Graphical Models for Machine Learning and DigitalCommunication.* MIT Press, Cambridge, Massachusetts, 1998.

[10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, Cambridge, Massachusetts, 2016.

[11] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-onApproach.* Morgan Kaufmann, 4th edition, 2022.

[12] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques.* MIT Press, Cambridge, Massachusetts, 2009.

[13] Maxim Lapan. *Deep Reinforcement Learning Hands-On.* Packt Publishing, 2nd edition, 2020.

[14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms.* Cambridge University Press, Cambridge, 2003.

[15] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall, 2008.

[16] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation.* Oxford University Press, Oxford, 2009.

[17] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics.* Oxford University Press, Oxford, 1999.

[18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information.* Cambridge University Press, Cambridge, 10th anniversary edition,2010.

[19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization.* Springer, 2nd edition, 2006.

[20] Antonio Ortega. *Introduction to Graph Signal Processing.* Cambridge University Press, Cambridge, 2022.

[21] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology.* Cambridge University Press, Cambridge, 2005.

[22] Simon J. D. Prince. *Understanding Deep Learning.* MIT Press, Cambridge, Massachusetts, 2023.

[23] Sebastian Raschka. *Build a Large Language Model (From Scratch).* Manning Publications, 2024.

[24] Kenneth H. Rosen. *Discrete Mathematics and Its Applications.* McGraw-Hill, 8th edition, 2019.

[25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.